

# Documentazione - Deck-BuilderCR

Vito Francesco Maistrini - Giovanni De Caro - Mauro Gorga

Gennaio 2026



## Indice

|          |                                                                                               |          |
|----------|-----------------------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Definizione del problema</b>                                                               | <b>2</b> |
| 1.1      | Introduzione . . . . .                                                                        | 2        |
| 1.2      | Obiettivi . . . . .                                                                           | 3        |
| 1.3      | Specifica PEAS . . . . .                                                                      | 3        |
| 1.3.1    | Caratteristiche dell'ambiente . . . . .                                                       | 4        |
| 1.4      | Analisi del problema: . . . . .                                                               | 4        |
| 1.5      | Assunzioni e limitazioni del dominio . . . . .                                                | 5        |
| 1.6      | Caratteristiche del gioco . . . . .                                                           | 6        |
| 1.6.1    | Caratteristiche delle carte . . . . .                                                         | 6        |
| 1.6.2    | Caratteristiche degli stati . . . . .                                                         | 6        |
| <b>2</b> | <b>Scelte tecniche progettuali: modellazione del problema</b>                                 | <b>7</b> |
| 2.1      | Rappresentazione della soluzione . . . . .                                                    | 7        |
| 2.2      | Vincoli del problema . . . . .                                                                | 7        |
| 2.2.1    | Vincoli imposti dall'utente . . . . .                                                         | 7        |
| 2.2.2    | Vincoli interni al gioco . . . . .                                                            | 8        |
| 2.2.3    | Vincoli interni del programma <i>Deck-BuilderCR</i> . . . . .                                 | 8        |
| <b>3</b> | <b>Scelte tecniche progettuali: funzione di fitness</b>                                       | <b>8</b> |
| 3.1      | Funzione di fitness adottata . . . . .                                                        | 9        |
| 3.2      | Qualità di attacco e difesa . . . . .                                                         | 9        |
| 3.2.1    | Scelta delle variabili per le componenti $Q_{\text{attacco}}$ e $Q_{\text{difesa}}$ . . . . . | 11       |

|          |                                                                                   |           |
|----------|-----------------------------------------------------------------------------------|-----------|
| 3.2.2    | Scelta dei parametri per le componenti $Q_{\text{attacco}}$ e $Q_{\text{difesa}}$ | 12        |
| 3.3      | Penalità di bilanciamento                                                         | 12        |
| 3.3.1    | Scelta delle variabili per la componente $P_{\text{bilanciamento}}$               | 13        |
| 3.3.2    | Scelta dei parametri della funzione di penalità                                   | 14        |
| 3.4      | Limiti della funzione di fitness                                                  | 14        |
| <b>4</b> | <b>Scelte tecniche progettuali: definizione degli algoritmi</b>                   | <b>14</b> |
| 4.1      | Inquadramento del problema                                                        | 14        |
| 4.2      | Algoritmi considerati                                                             | 14        |
| 4.3      | Hill Climbing                                                                     | 15        |
| 4.3.1    | Struttura dei vicini                                                              | 15        |
| 4.3.2    | Condizioni di arresto                                                             | 15        |
| 4.3.3    | Motivazioni dello scarto dell'algoritmo Hill Climbing                             | 16        |
| 4.4      | Algoritmo Genetico                                                                | 16        |
| 4.4.1    | Rappresentazione degli individui                                                  | 16        |
| 4.4.2    | Inizializzazione della popolazione                                                | 17        |
| 4.4.3    | Operatore di selezione                                                            | 17        |
| 4.4.4    | Operatore di crossover                                                            | 17        |
| 4.4.5    | Operatore di mutazione                                                            | 18        |
| <b>5</b> | <b>Interfaccia Grafica</b>                                                        | <b>18</b> |
| 5.1      | Configurazione per l'utente finale                                                | 18        |
| <b>6</b> | <b>Risultati e conclusione</b>                                                    | <b>19</b> |
| 6.1      | Fitness media                                                                     | 19        |
| 6.2      | Velocità di esecuzione                                                            | 19        |
| 6.3      | Varianza dei risultati                                                            | 19        |
| 6.4      | Conclusione                                                                       | 20        |
| <b>7</b> | <b>Disponibilità del codice sorgente</b>                                          | <b>20</b> |

# 1 Definizione del problema

## 1.1 Introduzione

Il progetto Deck-BuilderCR nasce per ovviare al problema della costruzione di un deck in Clash Royale che consiste nel selezionare un insieme limitato di carte (8), a partire da un insieme molto ampio di possibilità (truppe/edifici/incantesimi), dove ognuna ha un proprio costo di elisir (che varia dall'1 al 9).

La complessità del problema deriva dal fatto che lo spazio delle possibili combinazioni cresce in maniera combinatoria all'aumentare del numero di carte presenti nel gioco, e che la qualità del deck non dipende solo dalle singole carte, ma:

- Dall'efficienza della combinazione tra le carte
- Dal costo medio dell'elisir del deck

- Dalla Diversificazione della tipologia delle carte (truppe/edifici/incantesimi)

Dunque, il progetto Deck-BuilderCR è stato pensato principalmente per rivolgersi a dei giocatori alle prime armi e aiutarli a costruire deck bilanciati sotto tutti i vari punti di vista del gioco; questo dato che le carte presenti nel gioco sono tante e le combinazioni possibili per un deck raggiungono un numero spropositato.

## 1.2 Obiettivi

L'obiettivo principale del progetto Deck-BuilderCR, è la creazione di un sistema di IA in grado di generare automaticamente deck ottimizzati per Clash Royale, rispettando vincoli strutturali imposti dall'utente e massimizzando una misura di qualità del deck in base a dei vincoli interni del programma. In particolare, il sistema deve:

- Generare deck composti esattamente da 8 carte
- Garantire la coerenza del deck con dei vincoli selezionati dall'utente (tra quelli disponibili)
- Rispettare vincoli imposti dal gioco e dal sistema
- Massimizzare una funzione di valutazione che tenga conto della qualità complessiva del deck
- Fornire un output comprensibile all'utente finale tramite una dashboard che mostri il deck generato e ne spieghi le caratteristiche principali

## 1.3 Specifica PEAS

La specifica PEAS (Performance, Environment, Actuators, Sensors) viene utilizzata per descrivere formalmente il contesto operativo dell'agente.

**Performance (misure di prestazione):** Le metriche utilizzate per valutare l'operato dell'agente sono:

- Qualità complessiva del deck generato, misurata tramite una funzione di fitness multi-criterio soggetta a vincoli.
- Rispetto dei vincoli imposti, quali costo medio in elisir, carte fissate dall'utente, bilanciamento dei ruoli e tipologia delle truppe.
- Tempo di esecuzione e risorse computazionali utilizzate dall'algoritmo.

**Environment (ambiente):** L'ambiente in cui opera l'agente è costituito da:

- L'insieme delle carte disponibili con le rispettive caratteristiche.
- Le regole di composizione di un deck, che prevedono un numero fisso di 8 carte.

**Actuators (attuatori):** Le azioni che l'agente può intraprendere sono:

- Selezione di una carta da includere nel deck.
- Sostituzione di una carta con un'altra carta disponibile.

**Sensors (sensori):** Le informazioni percettive disponibili all'agente sono:

- Caratteristiche delle singole carte (costo, ruolo, tipologia, statistiche).
- Costo medio in elisir del deck.
- Copertura dei ruoli fondamentali nel deck.
- Valore della funzione di valutazione associata al deck corrente.

### 1.3.1 Caratteristiche dell'ambiente

Il problema può essere classificato secondo le seguenti proprietà dell'ambiente:

**Singolo agente:** Il problema è modellato come un problema a singolo agente, in cui non sono presenti altri agenti o avversari espliciti durante il processo di ricerca.

**Completamente osservabile:** L'ambiente è completamente osservabile, poiché l'agente ha accesso a tutte le informazioni rilevanti sulle carte disponibili, sui vincoli di composizione del deck e sul valore della funzione di valutazione.

**Deterministico:** L'ambiente è deterministico: a una data configurazione del deck e a una specifica azione (ad esempio la sostituzione di una carta) corrisponde sempre lo stesso stato successivo.

**Sequenziale:** Il processo di ricerca è sequenziale e consiste in una serie di trasformazioni dello stato finalizzate al miglioramento progressivo della soluzione. Le decisioni prese influenzano gli stati futuri.

**Statico:** L'ambiente è statico durante l'esecuzione dell'algoritmo, poiché le caratteristiche delle carte e i vincoli di composizione non variano nel tempo.

**Discreto:** Lo spazio degli stati è discreto, in quanto ogni deck è composto da un numero finito di carte selezionate da un insieme discreto.

## 1.4 Analisi del problema:

Il problema può essere formalizzato come un problema di ricerca in uno spazio degli stati, in cui ogni stato rappresenta un possibile deck valido o parzialmente valido.

**Stati:** Ogni stato rappresenta una configurazione di deck composta da un sottoinsieme di 8 carte.

**Stato iniziale:** Lo stato iniziale è un deck generato casualmente oppure un deck parzialmente definito dall'utente, il quale può fissare una o più carte.

**Azioni:** L'agente può eseguire due tipologie di azioni:

- Aggiunta di una carta al deck.
- Sostituzione di una carta con un'altra carta disponibile.

**Spazio degli stati:** Lo spazio degli stati è costituito da tutte le possibili combinazioni di 8 carte selezionabili dall'insieme totale delle carte disponibili.

**Test obiettivo:** L'obiettivo è la massimizzazione della qualità del deck secondo una funzione di valutazione (fitness function).

Il problema presenta le seguenti caratteristiche:

- Spazio degli stati estremamente ampio.
- Assenza di una soluzione ottima facilmente calcolabile in modo esatto.
- Presenza di molteplici vincoli da soddisfare simultaneamente.
- Necessità di bilanciare la qualità della soluzione con il costo computazionale.

A dicembre 2025 Clash Royale mette a disposizione 121 carte, per un numero di deck possibili pari a 899,749,479,915. Pertanto il problema non è risolvibile tramite ricerca esaustiva ed è più adatto all'impiego di tecniche di ricerca approssimata e ottimizzazione, come gli algoritmi genetici, che consentono di esplorare efficacemente lo spazio delle soluzioni senza enumerarlo completamente.

## 1.5 Assunzioni e limitazioni del dominio

Per il progetto *Deck-BuilderCR*, il dominio del problema è stato intenzionalmente semplificato introducendo alcune assunzioni e limitazioni, al fine di rendere il problema trattabile e di concentrarsi sugli aspetti di ricerca e ottimizzazione. In particolare, non sono state prese in considerazione le evoluzioni delle carte, i campioni e gli eroi (introdotti recentemente nel gioco). Tali elementi presentano meccaniche avanzate e dinamiche di gioco complesse, che richiederebbero un modello di valutazione più articolato e una rappresentazione dello stato significativamente più estesa. Di conseguenza, il numero totale di carte considerate nel dominio è pari a 113.

L'esclusione di queste componenti consente di:

- Ridurre la dimensionalità dello spazio degli stati.
- Focalizzare l'analisi sull'efficacia degli algoritmi di ricerca adottati.

## 1.6 Caratteristiche del gioco

### 1.6.1 Caratteristiche delle carte

Una carta rappresenta un componente elementare della soluzione (deck). Ogni carta è descritta mediante i seguenti attributi:

- **Danno/s:** danno inflitto al secondo.
- **Punti vita:** danni necessari per eliminare la carta.
- **Costo:** costo in elisir per schierare la carta.
- **Volante:** valore booleano che indica se la truppa non può essere colpita da unità da mischia.
- **Tipologia:** truppa, incantesimo o edificio.
- **Velocità:** velocità di movimento discretizzata (*lento*=0, *medio*=1, *rapido*=2, *molto rapido*=3).
- **Portata:** valore booleano, pari a 1 se la carta colpisce unità terrestri e volanti, 0 se colpisce solo unità terrestri.
- **Tipo di bersaglio:** valore booleano, pari a 1 se il bersaglio è esclusivamente la torre, 0 se il bersaglio include sia truppe sia torri.
- **Effetti aggiuntivi:** valore numerico intero che rappresenta la potenza degli effetti speciali della carta, compreso tra 0 e 2 (ad esempio congelamento, rallentamento, generazione di truppe, danni ad area, scudi).

La valutazione è un'euristica empirica utilizzata per stimare in modo approssimato il contributo strategico della carta. Il valore assegnato agli effetti aggiuntivi è in parte soggettivo e dipende dall'esperienza di gioco.

### 1.6.2 Caratteristiche degli stati

Uno stato è rappresentato da una configurazione di 8 carte (deck). Ogni deck è descritto dai seguenti attributi aggregati:

- **Danno medio/s:** media aritmetica del danno al secondo delle carte.
- **Punti ferita medi:** media aritmetica dei punti vita delle carte (escludendo gli incantesimi).
- **Costo medio:** media aritmetica del costo in elisir delle carte.
- **Numero di truppe volanti:** numero di carte di tipo truppa volante.
- **Numero di edifici:** numero di carte di tipologia edificio.
- **Numero di incantesimi:** numero di carte di tipologia incantesimo.

- **Velocità media:** media aritmetica della velocità delle carte.
- **Numero di carte con portata:** numero di carte che colpiscono unità terrestri e volanti.
- **Numero di carte con bersaglio torre:** numero di carte il cui bersaglio principale è la torre.
- **Effetti aggiuntivi totali:** somma dei valori degli effetti aggiuntivi delle carte del deck.

## 2 Scelte tecniche progettuali: modellazione del problema

### 2.1 Rappresentazione della soluzione

Una soluzione del problema è rappresentata da un deck composto da esattamente 8 carte selezionate dall'insieme delle carte disponibili, tale da rispettare i vincoli imposti e massimizzare una funzione di valutazione.

Ogni soluzione è modellata come un vettore di lunghezza fissa pari a 8:

$$s = (c_1, c_2, \dots, c_8)$$

dove ciascuna posizione  $c_i$  contiene una carta distinta selezionata dall'insieme delle carte disponibili. L'ordine delle posizioni non ha significato semantico, poiché un deck rappresenta una collezione non ordinata di carte.

Ogni carta è descritta tramite il proprio insieme di attributi, come costo in elisir, tipologia, statistiche e effetti speciali.

### 2.2 Vincoli del problema

Il problema è soggetto a tre categorie principali di vincoli:

- Vincoli imposti dall'utente.
- Vincoli interni del gioco.
- Vincoli interni del programma *Deck-BuilderCR*.

#### 2.2.1 Vincoli imposti dall'utente

L'utente può opzionalmente fissare un numero prefissato di carte da includere nel deck, variabile da 1 a 4. Tali carte devono essere necessariamente presenti in tutte le soluzioni candidate generate dall'algoritmo.

### 2.2.2 Vincoli interni al gioco

I vincoli imposti dal gioco sono i seguenti:

- Il deck deve contenere esattamente 8 carte.
- Tutte le carte del deck devono essere distinte (assenza di duplicati).

### 2.2.3 Vincoli interni del programma *Deck-BuilderCR*

Per evitare la generazione di deck fortemente sbilanciati, sono stati introdotti ulteriori vincoli euristici a livello di sistema. In particolare:

- Il deck può contenere al massimo 2 carte di tipologia incantesimo.

Questi vincoli sono stati introdotti per scartare configurazioni quasi inutilizzabili dal punto di vista strategico.

## 3 Scelte tecniche progettuali: funzione di fitness

La funzione di fitness è utilizzata per assegnare un punteggio a ciascuna soluzione candidata e valutare la qualità di un deck. Una prima versione della funzione era una combinazione lineare degli attributi dello stato. Tuttavia, tale approccio si è rivelato inadeguato, in quanto tendeva a privilegiare deck con costo medio elevato, le cui carte presentano statistiche di attacco e difesa molto alte, generando soluzioni poco realistiche dal punto di vista strategico.

Esempio di assegnamento di valori della funzione di fitness lineare:



Figura 1: DECK 1: Soluzione pessima, deck sbilanciato e troppo costoso (fitness = 188).





Figura 2: DECK 2: Soluzione buona, deck equilibrato e con costo moderato (fitness = 180).

### 3.1 Funzione di fitness adottata

La funzione di fitness adottata è una funzione non lineare basata su tre componenti principali:

$$F(s) = Q_{\text{attacco}}(s) + Q_{\text{difesa}}(s) - P_{\text{bilanciamento}}(s)$$

dove:

- $Q_{\text{attacco}}(s)$  misura la qualità offensiva del deck,
- $Q_{\text{difesa}}(s)$  misura la qualità difensiva del deck,
- $P_{\text{bilanciamento}}(s)$  rappresenta una penalità per configurazioni sbilanciate.

La scelta che ha portato alla creazione di questa funzione di fitness, è stata quella di basarsi sulla conoscenza del dominio applicativo degli sviluppatori. Inoltre la funzione di fitness definita in questo capitolo viene utilizzata come criterio di valutazione all'interno degli algoritmi di ricerca e ottimizzazione descritti nel capitolo successivo.

### 3.2 Qualità di attacco e difesa

La qualità offensiva è definita come:

$$Q_{\text{attacco}} = \sqrt{\text{DannoMedio}} \cdot (1 + 0.2 \cdot \text{NumeroTorre}) \cdot (1 + 0.2 \cdot \text{VelocitàMedia})$$

La qualità difensiva è definita come:

$$Q_{\text{difesa}} = \sqrt{\frac{\text{VitaMedia}}{6}} \cdot (1 + 0.1 \cdot \text{NumeroPortata}) \cdot (1 + 0.05 \cdot \text{EffettiAggiuntivi})$$

L'utilizzo della radice quadrata sulle variabili principali (*DannoMedio* e *VitaMedia*) consente di ridurre l'influenza delle feature con dominio numerico elevato, introducendo una normalizzazione non lineare che limita l'effetto di valori estremi e migliora la stabilità della funzione di fitness.

Il dominio dei valori di *DannoMedio* e *VitaMedia* è significativamente differente, come mostrato nella *Figura 3*. Per garantire un bilanciamento equilibrato tra le due variabili, la variabile *VitaMedia* viene normalizzata sostituendola con:

$$\frac{\text{VitaMedia}}{6}$$

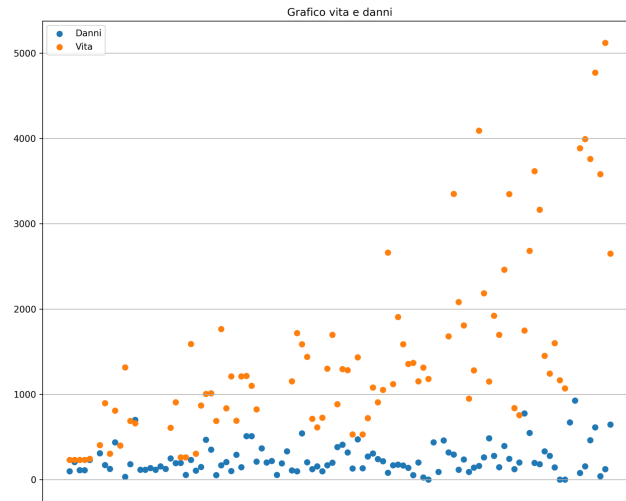
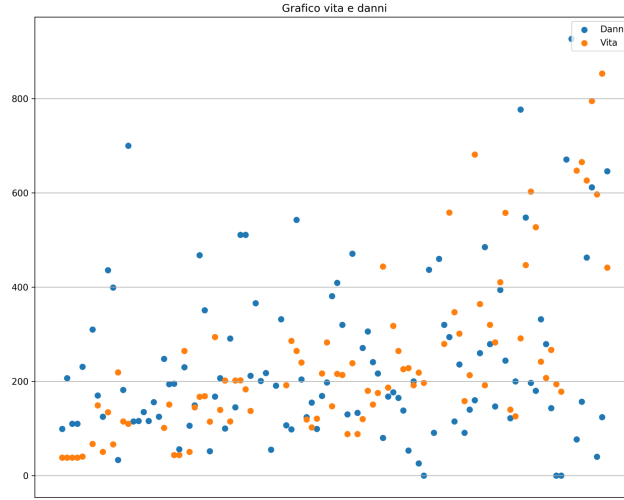


Figura 3: Distribuzione dei valori di *DannoMedio* e *VitaMedia*

Figura 4: Distribuzione normalizzata dei valori di VitaMedia



### 3.2.1 Scelta delle variabili per le componenti $Q_{\text{attacco}}$ e $Q_{\text{difesa}}$

Le variabili utilizzate per le componenti  $Q_{\text{attacco}}$  e  $Q_{\text{difesa}}$  sono state selezionate sulla base della conoscenza del dominio di Clash Royale e dell'analisi delle caratteristiche strategicamente rilevanti per la valutazione di un deck.

**Variabili per  $Q_{\text{attacco}}$  :** La componente offensiva è modellata tramite le seguenti variabili:

- *DannoMedio*: rappresenta il danno medio al secondo delle carte del deck ed è la principale misura della capacità offensiva.
- *NumeroTorre*: indica il numero di carte il cui bersaglio principale è la torre, modellando la presenza di win condition e la capacità di infliggere danni diretti alle strutture avversarie.
- *VelocitàMedia*: rappresenta la velocità media delle truppe nel deck ed è utilizzata per modellare la pressione offensiva e la rapidità di esecuzione delle strategie di attacco.

**Variabili per  $Q_{\text{difesa}}$  :** La componente difensiva è modellata tramite le seguenti variabili:

- *VitaMedia*: rappresenta la quantità media di punti vita delle truppe e degli edifici nel deck ed è una misura della capacità di resistenza difensiva.
- *NumeroPortata*: indica il numero di carte in grado di colpire unità sia volanti che terrestri, modellando la capacità di controllo del campo di battaglia.

- *EffettiAggiuntivi*: rappresenta un punteggio aggregato degli effetti speciali delle carte (ad esempio congelamento, rallentamento, generazione di truppe e danni ad area), che contribuiscono al controllo e alla gestione delle minacce avversarie.

### 3.2.2 Scelta dei parametri per le componenti $Q_{\text{attacco}}$ e $Q_{\text{difesa}}$

I parametri numerici utilizzati nelle componenti  $Q_{\text{attacco}}$  e  $Q_{\text{difesa}}$  sono stati selezionati sulla base di un euristica empirica data dalla conoscenza del dominio e da una fase preliminare di sperimentazione.

In particolare, i coefficienti moltiplicativi associati alle variabili *NumeroTorre*, *VelocitàMedia*, *NumeroPortata* ed *EffettiAggiuntivi* (sono rispettivamente 0.2, 0.2, 0.1 e 0.05)

### 3.3 Penalità di bilanciamento

La penalità per il bilanciamento del deck è definita come:

$$P_{\text{bilanciamento}} = p_{\text{costo}} + p_{\text{bersaglio}} + p_{\text{volante}} + p_{\text{portata}} + p_{\text{incantesimi}} + p_{\text{edifici}}$$

Le singole penalità sono definite come segue:

$$p_{\text{costo}} = (\text{CostoMedio} - 1)^3$$

$$p_{\text{bersaglio}} = \begin{cases} 3 \cdot \text{NumeroTorre} & \text{se NumeroTorre} > 2 \\ 0 & \text{altrimenti} \end{cases}$$

$$p_{\text{portata}} = \begin{cases} 15 & \text{se NumeroPortata} = 0 \\ 0 & \text{altrimenti} \end{cases}$$

$$p_{\text{incantesimi}} = \begin{cases} 12 & \text{se NumeroIncantesimi} = 0 \\ 0 & \text{altrimenti} \end{cases}$$

$$p_{\text{volante}} = \begin{cases} 10 & \text{se NumeroVolanti} = 0 \\ 4 & \text{se NumeroVolanti} > 3 \\ 0 & \text{altrimenti} \end{cases}$$

$$p_{\text{edifici}} = \begin{cases} 5 & \text{se NumeroEdifici} = 0 \\ 12 & \text{se NumeroEdifici} > 2 \\ 0 & \text{altrimenti} \end{cases}$$

### 3.3.1 Scelta delle variabili per la componente $P_{\text{bilanciamento}}$

Le variabili utilizzate nella componente di penalità  $P_{\text{bilanciamento}}$  sono state selezionate sulla base della conoscenza del dominio di Clash Royale e rappresentano alcune delle caratteristiche strategicamente più rilevanti nella costruzione di un deck efficace.

In particolare, tali variabili riflettono aspetti fondamentali del gioco, come il costo medio in elisir, la capacità di attaccare la torre, la copertura aerea, la possibilità di colpire unità volanti, la presenza di incantesimi e il supporto difensivo fornito dagli edifici. La scelta di queste variabili consente di penalizzare configurazioni fortemente sbilanciate che, pur presentando buone statistiche di attacco o difesa, risultano poco utilizzabili in una reale partita di gioco.

Di seguito vengono descritte le singole variabili che contribuiscono alla penalità di bilanciamento.

**Costo:** Il costo medio in elisir rappresenta uno degli aspetti più critici nella costruzione di un deck. Deck con costo medio troppo elevato risultano lenti e difficili da gestire durante una partita, penalizzando la capacità di risposta dell'utente. Per questo motivo, la variabile  $p_{\text{costo}}$  introduce una penalità crescente al crescere del costo medio del deck. Per la formula la decisione è stata quella di un'elevazione al cubo, per rendere il contributo della variabile più forte rispetto alle altre dato che è la più importante.

**Bersaglio:** La variabile relativa al numero di carte con bersaglio esclusivo sugli edifici e quindi come principale la torre, è utilizzata per evitare deck eccessivamente focalizzati sull'attacco diretto. Un numero elevato di tali carte riduce la capacità del deck di difendere efficacemente contro le truppe avversarie, rendendo la configurazione fragile dal punto di vista difensivo.

**Volante:** La presenza di truppe volanti è fondamentale per fronteggiare unità che non possono essere colpite da truppe terrestri, e per usare strategie di attacco più difficili da fronteggiare. L'assenza totale di truppe volanti, così come una loro presenza eccessiva, può generare forti squilibri. Per questo motivo, la penalità  $p_{\text{volante}}$  agisce sia in caso di assenza sia in caso di eccesso di truppe volanti.

**Incantesimi:** Gli incantesimi svolgono un ruolo chiave nel controllo del campo di battaglia, consentendo di eliminare gruppi di truppe o supportare le proprie unità. Un deck privo di incantesimi risulta generalmente meno flessibile, motivo per cui l'assenza totale di carte di questo tipo è penalizzata.

**Edifici:** Le carte di tipologia edificio forniscono supporto difensivo e controllo del campo. L'assenza di edifici riduce la capacità del deck di gestire unità avversarie pesanti, mentre un numero eccessivo di edifici limita le opzioni offensive. La penalità  $p_{\text{edifici}}$  è quindi applicata sia sull'assenza di questi che su un numero eccessivo nel deck.

### 3.3.2 Scelta dei parametri della funzione di penalità

I parametri numerici utilizzati nella componente  $P_{\text{bilanciamento}}$  non derivano da un modello teorico formale, ma sono stati determinati tramite un processo di euristica empirica basato sulla conoscenza del dominio di Clash Royale da parte degli sviluppatori.

In particolare, i valori assegnati alle singole penalità sono il risultato di considerazioni qualitative sulle dinamiche di gioco e di sperimentazioni preliminari volte a individuare configurazioni di deck realistiche ed efficaci. L'obiettivo principale nella scelta di tali parametri è stato quello di penalizzare in modo significativo le situazioni considerate critiche dal punto di vista strategico, senza tuttavia annullare completamente il contributo positivo delle componenti di attacco e difesa.

### 3.4 Limiti della funzione di fitness

La funzione di fitness proposta rappresenta una semplificazione del dominio di Clash Royale e non modella esplicitamente le sinergie tra carte né il meta-game competitivo. Inoltre, la valutazione è indipendente dall'avversario e dal contesto di partita, assumendo che la qualità del deck possa essere stimata in modo isolato.

## 4 Scelte tecniche progettuali: definizione degli algoritmi

### 4.1 Inquadramento del problema

Il problema della generazione automatica di deck di Clash Royale è riconducibile a un problema di ottimizzazione combinatoria caratterizzato da uno spazio degli stati estremamente ampio e dalla presenza di numerosi vincoli. In tale contesto, l'utilizzo di algoritmi di ricerca esatta risulta impraticabile, rendendo necessario l'impiego di tecniche di ricerca approssimata e metaeuristiche.

Gli algoritmi selezionati per questo progetto sono stati scelti con l'obiettivo di esplorare efficacemente lo spazio delle soluzioni e individuare configurazioni di elevata qualità in tempi computazionali bassi.

### 4.2 Algoritmi considerati

Nel contesto del progetto *Deck-BuilderCR* sono stati presi in considerazione principalmente due approcci:

- L'utilizzo dell'algoritmo *Hill Climbing* è stata la scelta iniziale, ma in secondo momento scartata per delle profonde lacune verso il dominio applicativo

- L'utilizzo di un *Algoritmo Genetico* è stata la soluzione finale presa in considerazione dopo aver scartato la soluzione che prevedeva l'algoritmo Hill Climbing

### 4.3 Hill Climbing

L'algoritmo di *Hill Climbing* è uno dei più semplici algoritmi di ricerca locale e ha l'obiettivo di migliorare iterativamente una soluzione corrente esplorando il suo intorno nello spazio di ricerca, fino al raggiungimento di un massimo (o minimo) locale della funzione di fitness.

Nonostante la sua semplicità, l'Hill Climbing è stato inizialmente considerato una possibile soluzione per il problema affrontato, in quanto il dominio non presenta un vero e proprio ottimo globale assoluto. Infatti, nel contesto di Clash Royale, la qualità di un deck non è determinata unicamente da criteri oggettivi, ma dipende anche da preferenze soggettive dei giocatori e da stili di gioco differenti. Tuttavia, come verrà discusso nelle sezioni successive, l'approccio basato su Hill Climbing presenta limitazioni significative nel dominio considerato, che ne hanno motivato lo scarto come soluzione finale.

#### 4.3.1 Struttura dei vicini

La struttura dei vicini è definita tramite una funzione:

$$N : \mathcal{S} \rightarrow \mathcal{P}(\mathcal{S})$$

dove:

- $\mathcal{S}$  rappresenta l'insieme di tutte le soluzioni possibili, ovvero l'insieme di tutti i deck creabili a partire dalle carte disponibili;
- $\mathcal{P}(\mathcal{S})$  indica l'insieme delle parti di  $\mathcal{S}$ ;
- $N(s) \subseteq \mathcal{S}$  è l'insieme delle soluzioni vicine allo stato  $s$ .

Nel contesto del progetto, due soluzioni sono considerate vicine se differiscono per una singola carta. In particolare, una soluzione vicina è ottenuta sostituendo una delle 8 carte del deck corrente con una carta diversa selezionata dall'insieme delle carte disponibili.

Assumendo la presenza di 113 carte totali e l'assenza di duplicati, ogni stato presenta fino a  $8 \times 105 = 840$  soluzioni vicine possibili.

#### 4.3.2 Condizioni di arresto

Nel progetto, l'algoritmo di Hill Climbing termina in una delle seguenti condizioni:

1. non esiste una soluzione vicina con valore di fitness maggiore rispetto alla soluzione corrente:

$$\nexists s' \in N(s) : f(s') > f(s)$$

2. viene raggiunto un numero massimo prefissato di iterazioni pari a 300.

La seconda condizione è stata introdotta per evitare che l'algoritmo impieghi un tempo eccessivo nel tentativo di migliorare una soluzione che non presenta ulteriori miglioramenti significativi.

Il valore massimo di 300 iterazioni è stato determinato empiricamente a seguito di test sperimentali con valori inferiori e superiori. Tale scelta rappresenta un compromesso (*trade-off*) tra efficienza computazionale e qualità della soluzione ottenuta.

### 4.3.3 Motivazioni dello scarto dell'algoritmo Hill Climbing

Nonostante la semplicità implementativa e le soluzioni con alta fitness che dava, l'algoritmo di Hill Climbing si è rivelato non adeguato al dominio applicativo considerato.

In particolare, il problema della generazione di deck di Clash Royale presenta uno spazio delle soluzioni altamente multimodale, caratterizzato dalla presenza di numerosi ottimi locali; inoltre come fattore più importante, abbiamo che il problema della generazione di un deck di Clash Royale non ha una soluzione ottima globale definitiva, ma ci sono varie soluzioni ottime in base anche alle preferenze soggettive del giocatore. In tale contesto, l'Hill Climbing tende a convergere rapidamente verso soluzioni localmente ottimali, senza la capacità di esplorare regioni alternative dello spazio di ricerca.

Inoltre, l'algoritmo non garantisce un'adeguata diversità delle soluzioni generate, producendo spesso deck molto simili tra loro, se non addirittura quasi sempre uguali e limitando la possibilità di individuare configurazioni significativamente differenti ma potenzialmente più efficaci.

Queste limitazioni hanno portato allo scarto dell'Hill Climbing come strategia principale, rendendo necessario l'utilizzo di un approccio in grado di bilanciare efficacemente esplorazione e sfruttamento dello spazio delle soluzioni, producendo quasi sempre soluzioni diverse che possano soddisfare

## 4.4 Algoritmo Genetico

A seguito dello scarto dell'algoritmo di Hill Climbing, è stato adottato come soluzione finale un approccio basato sulla famiglia degli *Algoritmi Genetici*. Tale scelta è motivata dalla necessità di esplorare in modo più efficace uno spazio delle soluzioni ampio, complesso e fortemente multimodale, riducendo il rischio di convergenza prematura verso ottimi locali.

L'Algoritmo Genetico consente infatti di mantenere una popolazione di soluzioni candidate e di introdurre diversità nel processo di ricerca tramite operatori stocastici, risultando particolarmente adatto al dominio applicativo considerato

### 4.4.1 Rappresentazione degli individui

Ogni individuo della popolazione rappresenta una possibile soluzione del problema ed è modellato come un vettore di geni di lunghezza fissa pari a 8:



$$I = (g_1, g_2, \dots, g_8)$$

dove ciascun gene  $g_i$  rappresenta una carta del deck.

Il dominio dei geni è costituito dall'insieme delle carte disponibili nel gioco, escludendo i Campioni, in accordo con le assunzioni e le limitazioni del dominio definite nel Capitolo 1.

#### 4.4.2 Inizializzazione della popolazione

La popolazione iniziale è generata in modo completamente casuale, con una dimensione pari a 100 individui. Durante la fase di inizializzazione, possono essere applicati eventuali vincoli imposti dall'utente, come la presenza obbligatoria di alcune carte nel deck.

La dimensione del *mating pool* invece è leggermente inferiore, ed è 80, garantendo che quasi tutti gli individui possano partecipare al processo di crossover.

La scelta di un'inizializzazione casuale è motivata dalle caratteristiche del dominio di Clash Royale, nel quale esistono deck efficaci basati su combinazioni molto differenti di carte. Un approccio casuale consente quindi di introdurre fin dall'inizio una buona diversità nella popolazione. Mentre la scelta di una *size population* di 100 e di un *mating pool* di 80, è stata presa dopo varie sperimentazioni che hanno portato alla conclusione che è meglio avere una dimensione di possibili soluzioni abbastanza ampia ma non troppo, per avere un giusto bilanciamento tra efficienza della soluzione finale e costo computazionale.

#### 4.4.3 Operatore di selezione

Come operatore di selezione è stato adottato il metodo di *Rank Selection*. In questo approccio, gli individui vengono ordinati in base al valore di fitness e la probabilità di selezione dipende dalla posizione in classifica piuttosto che dal valore assoluto della fitness.

Questa scelta consente di ridurre la pressione selettiva e di preservare la diversità della popolazione, favorendo una migliore esplorazione dello spazio delle soluzioni, a fronte di un lieve aumento del costo computazionale dovuto all'ordinamento (*trade-off*).

#### 4.4.4 Operatore di crossover

L'operatore di crossover scelto è il *Uniform Crossover*. Durante il crossover, ciascun gene del figlio viene selezionato casualmente da uno dei due genitori.

Nel caso in cui il deck generato risulti non valido, l'operazione di crossover viene ripetuta per un numero massimo prefissato di tentativi. Se anche dopo tali tentativi il figlio risulta invalido, viene restituito uno dei due genitori selezionato casualmente.

La probabilità di crossover è fissata pari a 1, in modo da favorire una costante ricombinazione genetica e una migliore esplorazione dello spazio degli stati. Nel

raro caso in cui il crossover non venga applicato, l'individuo restituito è uno dei due genitori scelto casualmente.

#### 4.4.5 Operatore di mutazione

Come operatore di mutazione è stato adottato il *Random Resetting*. In questo approccio, un gene viene selezionato casualmente e sostituito con un'altra carta disponibile, in modo tale da non violare i vincoli di validità del deck.

La probabilità di mutazione è fissata a 0.8. La scelta di avere una probabilità di mutazione così alta consente di avere molta casualità, che per il dominio del problema è fondamentale.

## 5 Interfaccia Grafica

L'interfaccia grafica sviluppata ha lo scopo di facilitare l'interazione dell'utente con il progetto nel suo complesso. Attraverso il menù principale è possibile selezionare il tipo di algoritmo da utilizzare, scegliendo tra Hill Climbing e Algoritmo Genetico, e configurarne i rispettivi parametri, permettendo così all'utente di personalizzare l'esecuzione dell'algoritmo.

Inoltre, l'interfaccia consente di selezionare fino a quattro carte iniziali che il mazzo finale dovrà obbligatoriamente contenere, vincolando in questo modo il processo di generazione del deck.

Nella schermata principale vengono mostrati i risultati finali dell'esecuzione, includendo la fitness ottenuta e le principali statistiche relative alle carte del mazzo generato, calcolate in base ai parametri selezionati dall'utente. È inoltre presente una sezione dedicata allo storico delle esecuzioni, nella quale sono memorizzate tutte le generazioni prodotte dall'algoritmo, consentendo all'utente di consultarle in qualsiasi momento.

### 5.1 Configurazione per l'utente finale

Per eseguire correttamente l'applicazione e avviare l'interfaccia grafica, è necessario seguire i passaggi descritti di seguito, utilizzando il terminale dell'IDE (in questo caso PyCharm).

1. **Installazione delle dipendenze** (sia su macOS che su Windows):

```
pip install -r requirements.txt
```

2. **Inizializzazione dell'ambiente virtuale:**

*Su macOS:*

```
source .venv/bin/activate
```

*Su Windows:*

```
.venv\Scripts\activate
```

### 3. Avvio dell'interfaccia grafica:

```
streamlit run DeckBuilderCR/GUI/guiClashRoyale.py
```

## 6 Risultati e conclusione

I risultati presentati in questo capitolo sono stati ottenuti eseguendo sia l'algoritmo genetico sia l'algoritmo di Hill Climbing per un totale di 50 esecuzioni indipendenti ciascuno. I parametri utilizzati per l'algoritmo genetico durante le fasi di test sono quelli descritti nel Capitolo 4.4.

### 6.1 Fitness media

L'analisi dei risultati mostra che l'algoritmo di Hill Climbing ottiene, in media, soluzioni con una fitness più elevata rispetto all'algoritmo genetico. In particolare, il GA produce mazzi con una fitness media pari a 47,3, mentre l'Hill Climbing raggiunge una fitness media di 57,9.

Questo risultato indica che, dal punto di vista puramente ottimizzativo, l'Hill Climbing tende a convergere verso soluzioni di qualità superiore. Tuttavia, tale comportamento deve essere valutato in relazione alle altre metriche considerate e agli obiettivi complessivi del problema.

### 6.2 Velocità di esecuzione

Per quanto riguarda le prestazioni temporali, l'algoritmo genetico risulta significativamente più rapido rispetto all'Hill Climbing. Il tempo medio di esecuzione dell'Hill Climbing è pari a 4535 ms per singola esecuzione, mentre l'algoritmo genetico impiega in media soltanto 125 ms.

Questo divario è dovuto alla natura iterativa e locale dell'Hill Climbing, che richiede numerosi passi di miglioramento successivo, a fronte di un GA che sfrutta operatori evolutivi in grado di esplorare lo spazio delle soluzioni in modo più efficiente dal punto di vista computazionale.

### 6.3 Varianza dei risultati

La varianza rappresenta una misura della dispersione dei risultati attorno al valore medio e fornisce indicazioni sulla stabilità e sulla diversità delle soluzioni generate.

Formalmente, data una serie di valori di fitness ottenuti da  $N$  esecuzioni indipendenti dell'algoritmo,

$$f_1, f_2, \dots, f_N,$$

la varianza è definita come:

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (f_i - \bar{f})^2$$

dove  $\bar{f}$  rappresenta la fitness media, calcolata come:

$$\bar{f} = \frac{1}{N} \sum_{i=1}^N f_i.$$

Nel nostro caso, l'algoritmo genetico presenta una varianza pari a 5,99, mentre l'Hill Climbing mostra una varianza prossima allo zero. Sebbene una varianza bassa sia spesso considerata un indicatore positivo di stabilità, nel contesto specifico del problema tale comportamento risulta limitante.

Una varianza prossima allo zero indica infatti che, indipendentemente dallo stato iniziale, l'Hill Climbing converge quasi sempre verso soluzioni molto simili tra loro, suggerendo una forte dipendenza dai massimi locali dello spazio degli stati. Al contrario, la varianza osservata nel GA, pur rimanendo contenuta, è sufficientemente elevata da indicare la produzione di soluzioni diverse tra loro, caratteristica desiderabile nel problema in esame.

## 6.4 Conclusione

Sulla base delle analisi effettuate, la soluzione finale adottata è l'algoritmo genetico. Come anticipato nel Capitolo 2, il GA risulta maggiormente coerente con gli obiettivi del problema, grazie alla sua maggiore flessibilità e alla capacità di esplorare in modo più efficace lo spazio degli stati.

Sebbene l'Hill Climbing produca soluzioni mediamente più ottime in termini di fitness, esso soffre di una forte convergenza verso soluzioni simili, riducendo la varietà dei mazzi generati. L'algoritmo genetico, invece, pur non garantendo sempre la soluzione ottima, è in grado di generare soluzioni costantemente buone e sufficientemente diverse tra loro, rendendolo più adatto al contesto applicativo considerato.

## 7 Disponibilità del codice sorgente

Il codice sorgente completo del progetto, comprensivo dell'implementazione degli algoritmi, dell'interfaccia grafica e dei file di configurazione, è disponibile pubblicamente al seguente indirizzo:

<https://github.com/girgio/Deck-BuilderCR>